

\$ pakai dashboard

## PakAI – AI Usage Tracker

Daemon 4062s | 4 providers | refreshed 1s ago

|    |                      |                        |        |            |                   |  |
|----|----------------------|------------------------|--------|------------|-------------------|--|
| oc | opencode/openai      |                        |        |            |                   |  |
|    | 5h                   | <div><div></div></div> | 1%     | 1% / 100%  |                   |  |
|    | w                    | <div><div></div></div> | 10%    | 10% / 100% | resets in 1d 16h  |  |
| cl | claude               |                        |        |            |                   |  |
|    | 5h                   | <div><div></div></div> | 26%    | 26% / 100% | resets in 1h 51m  |  |
|    | w                    | <div><div></div></div> | 10%    | 10% / 100% | resets in 19h 21m |  |
| oa | openai               |                        |        |            |                   |  |
|    | 5h                   | <div><div></div></div> | 1%     | 1% / 100%  |                   |  |
|    | w                    | <div><div></div></div> | 10%    | 10% / 100% | resets in 1d 16h  |  |
| og | opencode/opencode-go |                        |        |            |                   |  |
| m  |                      |                        | \$2.51 |            |                   |  |

q: quit    r: refresh    d: toggle debug

# pakai

pantau semua AI-mu, dari terminal

claude • codex • opencode

multi-provider • real-time • open source

Dian Hanifudin Subhi • Dhebys Suryani Hormansyah  
Agung Nugroho Pramudhita • Sofyan Noor Arief

v1.0 • Mei 2026

PakAI adalah alat bantu pemantau penggunaan langganan AI yang bersifat open-source dan gratis digunakan.

<https://github.com/dhanifudin/pakai>

# Daftar Isi

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Pendahuluan</b>                                     | <b>1</b>  |
| 1.1      | Apa itu PakAI? . . . . .                               | 1         |
| 1.2      | Fitur Utama . . . . .                                  | 1         |
| 1.3      | Tampilan Aplikasi . . . . .                            | 1         |
| 1.4      | Cara Kerja . . . . .                                   | 2         |
| <b>2</b> | <b>Persyaratan Sistem</b>                              | <b>3</b>  |
| 2.1      | Sistem Operasi . . . . .                               | 3         |
| 2.2      | Perangkat Lunak yang Dibutuhkan . . . . .              | 3         |
| 2.2.1    | Go (untuk build dari kode sumber) . . . . .            | 3         |
| 2.2.2    | Penyedia AI (opsional, sesuai kebutuhan) . . . . .     | 3         |
| 2.3      | Persyaratan Jaringan . . . . .                         | 4         |
| <b>3</b> | <b>Instalasi</b>                                       | <b>5</b>  |
| 3.1      | Metode 1: Install dari Go (Direkomendasikan) . . . . . | 5         |
| 3.2      | Metode 2: Build dari Kode Sumber . . . . .             | 5         |
| 3.2.1    | Kloning repositori . . . . .                           | 5         |
| 3.2.2    | Kompilasi . . . . .                                    | 5         |
| 3.2.3    | Pasang ke PATH . . . . .                               | 5         |
| 3.3      | Metode 3: Binary Release . . . . .                     | 6         |
| 3.4      | Verifikasi Instalasi . . . . .                         | 6         |
| <b>4</b> | <b>Konfigurasi Awal</b>                                | <b>7</b>  |
| 4.1      | Perintah Setup Otomatis . . . . .                      | 7         |
| 4.1.1    | Yang Dilakukan oleh <code>pakai setup</code> . . . . . | 7         |
| 4.2      | Memulai Daemon . . . . .                               | 7         |
| 4.2.1    | Memulai daemon secara manual . . . . .                 | 7         |
| 4.2.2    | Memeriksa status daemon . . . . .                      | 8         |
| 4.2.3    | Menghentikan daemon . . . . .                          | 8         |
| 4.3      | Konfigurasi Autostart dengan Systemd . . . . .         | 9         |
| 4.4      | Autentikasi Penyedia . . . . .                         | 9         |
| 4.4.1    | Claude . . . . .                                       | 9         |
| 4.4.2    | OpenAI Codex . . . . .                                 | 9         |
| 4.4.3    | OpenCode Go . . . . .                                  | 9         |
| <b>5</b> | <b>Penggunaan Dasar</b>                                | <b>11</b> |
| 5.1      | Mengapa Memantau Penggunaan AI? . . . . .              | 11        |
| 5.2      | Memeriksa Status Penggunaan . . . . .                  | 11        |
| 5.2.1    | Output JSON . . . . .                                  | 12        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 5.3      | Dashboard TUI Interaktif . . . . .                             | 12        |
| 5.3.1    | Membaca tampilan dashboard . . . . .                           | 12        |
| 5.3.2    | Kode warna . . . . .                                           | 13        |
| 5.3.3    | Kapan dashboard berguna . . . . .                              | 13        |
| 5.4      | Output untuk Tmux . . . . .                                    | 14        |
| 5.5      | Output untuk Waybar . . . . .                                  | 14        |
| 5.6      | Men-debug Penyedia . . . . .                                   | 15        |
| 5.7      | Completion Shell . . . . .                                     | 15        |
| <b>6</b> | <b>Integrasi</b>                                               | <b>17</b> |
| 6.1      | Integrasi Tmux . . . . .                                       | 17        |
| 6.1.1    | Konfigurasi dasar . . . . .                                    | 17        |
| 6.1.2    | Konfigurasi dengan tmux-powerline . . . . .                    | 17        |
| 6.1.3    | Persyaratan font . . . . .                                     | 17        |
| 6.2      | Integrasi Waybar . . . . .                                     | 18        |
| 6.2.1    | Konfigurasi modul . . . . .                                    | 18        |
| 6.2.2    | Konfigurasi CSS . . . . .                                      | 18        |
| 6.2.3    | Tooltip . . . . .                                              | 19        |
| 6.3      | Integrasi Shell Prompt . . . . .                               | 19        |
| <b>7</b> | <b>Konfigurasi Lanjutan</b>                                    | <b>21</b> |
| 7.1      | Berkas Konfigurasi . . . . .                                   | 21        |
| 7.2      | Melihat Semua Konfigurasi . . . . .                            | 21        |
| 7.3      | Mengubah Konfigurasi . . . . .                                 | 22        |
| 7.4      | Referensi Kunci Konfigurasi . . . . .                          | 22        |
| 7.4.1    | Konfigurasi Daemon . . . . .                                   | 22        |
| 7.4.2    | Konfigurasi Tampilan . . . . .                                 | 22        |
| 7.4.3    | Konfigurasi Ambang Batas . . . . .                             | 22        |
| 7.4.4    | Konfigurasi Per Penyedia . . . . .                             | 23        |
| 7.5      | Contoh Berkas Konfigurasi Lengkap . . . . .                    | 23        |
| 7.6      | Mock Provider (untuk Pengujian) . . . . .                      | 24        |
| <b>8</b> | <b>Referensi Perintah</b>                                      | <b>25</b> |
| 8.1      | Perintah Utama . . . . .                                       | 25        |
| 8.2      | Perintah Daemon . . . . .                                      | 25        |
| 8.3      | Perintah Setup . . . . .                                       | 26        |
| 8.4      | Perintah Konfigurasi . . . . .                                 | 26        |
| 8.5      | Perintah Penyedia . . . . .                                    | 26        |
| 8.6      | HTTP API Daemon . . . . .                                      | 26        |
| <b>9</b> | <b>Pemecahan Masalah</b>                                       | <b>29</b> |
| 9.1      | Masalah Instalasi . . . . .                                    | 29        |
| 9.1.1    | Perintah <code>pakai</code> tidak ditemukan . . . . .          | 29        |
| 9.1.2    | Build gagal: versi Go terlalu lama . . . . .                   | 29        |
| 9.2      | Masalah Daemon . . . . .                                       | 29        |
| 9.2.1    | Daemon gagal dimulai: port sudah digunakan . . . . .           | 29        |
| 9.2.2    | Daemon berhenti tiba-tiba . . . . .                            | 30        |
| 9.2.3    | Status daemon menunjukkan data lama ( <i>stale</i> ) . . . . . | 30        |
| 9.3      | Masalah Penyedia Claude . . . . .                              | 30        |

---

|       |                                                     |    |
|-------|-----------------------------------------------------|----|
| 9.3.1 | Claude stats cache tidak ditemukan . . . . .        | 30 |
| 9.3.2 | Kredensial Claude tidak ditemukan . . . . .         | 30 |
| 9.3.3 | Persentase Claude tidak tersedia . . . . .          | 30 |
| 9.4   | Masalah Penyedia OpenAI Codex . . . . .             | 31 |
| 9.4.1 | Kredensial Codex tidak ditemukan . . . . .          | 31 |
| 9.4.2 | Permintaan penggunaan Codex gagal . . . . .         | 31 |
| 9.4.3 | Kesalahan sertifikat TLS . . . . .                  | 31 |
| 9.5   | Masalah Penyedia OpenCode . . . . .                 | 31 |
| 9.5.1 | Database OpenCode tidak ditemukan . . . . .         | 31 |
| 9.5.2 | Database OpenCode sibuk ( <i>locked</i> ) . . . . . | 31 |
| 9.6   | Masalah Tampilan . . . . .                          | 32 |
| 9.6.1 | Ikon tidak muncul di tmux . . . . .                 | 32 |
| 9.6.2 | Waybar tidak memperbarui data . . . . .             | 32 |
| 9.7   | Diagnosis Umum . . . . .                            | 32 |



# Bab 1

## Pendahuluan

### 1.1 Apa itu PakAI?

**PakAI** (dijalankan dengan perintah `pakai`) adalah alat bantu berbasis terminal untuk memantau penggunaan langganan layanan AI secara terpadu. Alat ini menampilkan sisa kuota dan biaya untuk beberapa penyedia AI sekaligus dalam satu tampilan yang ringkas.

PakAI mendukung tiga penyedia layanan AI:

- **Claude** (Anthropic): memantau persentase penggunaan pesan per 5 jam, mingguan, dan bulanan melalui OAuth API.
- **OpenAI Codex**: memantau kuota penggunaan per 5 jam dan mingguan melalui OAuth API.
- **OpenCode Go**: memantau biaya penggunaan per penyedia dari database lokal SQLite.

### 1.2 Fitur Utama

- Pemantauan multi-penyedia dalam satu perintah.
- Tiga jendela waktu: 5 jam, mingguan, dan bulanan.
- Tampilan berwarna: hijau (aman), kuning (peringatan), merah (kritis).
- Empat mode tampilan: status CLI, tmux, Waybar, dan dashboard TUI.
- Daemon latar belakang dengan pembaruan otomatis dan API HTTP.
- Pengaturan melalui berkas konfigurasi TOML.
- Deteksi otomatis penyedia yang terinstal.

### 1.3 Tampilan Aplikasi

Gambar di bawah menampilkan output perintah `pakai status` yang menunjukkan ringkasan penggunaan semua penyedia yang terdeteksi.

```
$ pakai status

openai          0.00          (no limit set)
claude          5h: 7% (resets in 4h 25m) | w: 8% (resets in 21h 55m)
opencode/openai $0.00        (no limit set)
opencode/opencode-go $2.51    (no limit set)

4 providers · refreshed 19s ago
```

Gambar 1.1: Output `pakai status`: ringkasan penggunaan semua penyedia

## 1.4 Cara Kerja

PakAI berjalan sebagai *background daemon* yang secara berkala mengambil data penggunaan dari setiap penyedia. Data ini kemudian ditampilkan melalui berbagai antarmuka:

1. **Daemon** berjalan di port 7731 dan menyimpan data terbaru.
2. Perintah `pakai status`, `pakai tmux`, dan `pakai waybar` mengambil data dari daemon melalui HTTP.
3. `pakai dashboard` menampilkan antarmuka TUI interaktif dengan pembaruan langsung (*live update*).



# Bab 2

## Persyaratan Sistem

### 2.1 Sistem Operasi

PakAI dirancang untuk berjalan di lingkungan Unix dan Unix-like. Sistem yang didukung:

- **Linux** (Ubuntu, Debian, Fedora, Arch, NixOS, dan distribusi lainnya)
- **macOS** (Ventura, Sonoma, Sequoia, dan versi lebih baru)
- **WSL2** (Windows Subsystem for Linux 2) di Windows 10/11

### 2.2 Perangkat Lunak yang Dibutuhkan

#### 2.2.1 Go (untuk build dari kode sumber)

Diperlukan Go versi **1.21** atau lebih baru jika ingin melakukan kompilasi sendiri dari kode sumber.

```
$ go version
go version go1.25.0 linux/amd64
```

Unduh Go dari <https://go.dev/dl/> atau gunakan manajer versi seperti **asdf**, **mise**, atau **nix**.

#### 2.2.2 Penyedia AI (opsional, sesuai kebutuhan)

Pasang dan autentikasi penyedia yang ingin dipantau:

| Penyedia     | Perangkat       | Autentikasi                                                |
|--------------|-----------------|------------------------------------------------------------|
| Claude       | Claude Code CLI | <code>claude login</code>                                  |
| OpenAI Codex | Codex CLI       | <code>codex login</code>                                   |
| OpenCode Go  | OpenCode        | Langganan di <a href="https://opencode.ai">opencode.ai</a> |

Tidak perlu memasang semua penyedia. PakAI akan mendeteksi otomatis penyedia mana yang tersedia.

## 2.3 Persyaratan Jaringan

Semua penyedia yang didukung memerlukan koneksi internet:

- **Claude** dan **OpenAI Codex** memerlukan koneksi internet untuk mengambil data penggunaan melalui API OAuth.
- **OpenCode Go** memerlukan koneksi internet untuk menjalankan sesi OpenCode CLI. PakAI sendiri membaca data penggunaan dari database lokal yang dibuat oleh OpenCode, tetapi OpenCode CLI membutuhkan jaringan saat digunakan.

# Bab 3

## Instalasi

### 3.1 Metode 1: Install dari Go (Direkomendasikan)

Cara termudah adalah menggunakan `go install` untuk mengunduh dan mengkompilasi PakAI secara langsung:

```
$ go install github.com/dhanifudin/pakai/cmd/pakai@latest
```

Pastikan direktori `$GOPATH/bin` (biasanya `~/go/bin`) sudah ada di `$PATH`:

```
# Tambahkan ke ~/.bashrc atau ~/.zshrc
export PATH="$PATH:$HOME/go/bin"
```

### 3.2 Metode 2: Build dari Kode Sumber

#### 3.2.1 Kloning repositori

```
$ git clone https://github.com/dhanifudin/pakai.git
$ cd pakai
```

#### 3.2.2 Kompilasi

```
$ go build -ldflags="-s -w" -o pakai ./cmd/pakai
```

Atau gunakan Makefile yang sudah disediakan:

```
$ make build
```

#### 3.2.3 Pasang ke PATH

```
$ sudo mv pakai /usr/local/bin/
# atau ke direktori lokal tanpa sudo:
$ mv pakai ~/.local/bin/
```

### 3.3 Metode 3: Binary Release

Binary siap pakai tersedia di halaman *Releases* GitHub untuk arsitektur Linux AMD64 dan ARM64:

1. Kunjungi <https://github.com/dhanifudin/pakai/releases>
2. Unduh arsip yang sesuai, misalnya:  
`pakai_1.0.0_linux_amd64.tar.gz`
3. Ekstrak dan pindahkan binary:

```
$ tar -xzf pakai_1.0.0_linux_amd64.tar.gz
$ chmod +x pakai
$ mv pakai ~/.local/bin/
```

### 3.4 Verifikasi Instalasi

Setelah instalasi, mulai daemon lalu jalankan perintah status untuk memverifikasi PakAI berjalan dengan benar:

```
$ pakai daemon start
$ pakai status
```

```
$ pakai status

openai          0.00      (no limit set)
claude          5h: 7% (resets in 4h 25m) | w: 8% (resets in 21h 55m)
opencode/openai $0.00    (no limit set)
opencode/opencode-go $2.51 (no limit set)

4 providers · refreshed 19s ago
```

Gambar 3.1: Output `pakai status` setelah instalasi berhasil

Jika perintah tidak ditemukan, pastikan direktori instalasi ada di `$PATH`.

#### Perhatian

Jika menggunakan **asdf** atau **mise** sebagai manajer versi Go, pastikan shim untuk **pakai** sudah diperbarui setelah instalasi. Jalankan `asdf reshim golang` atau `mise reshim` setelah proses instalasi.

# Bab 4

## Konfigurasi Awal

### 4.1 Perintah Setup Otomatis

PakAI menyediakan perintah `pakai setup` yang mendeteksi penyedia yang terinstal, menulis unit systemd, dan menampilkan cuplikan konfigurasi yang siap digunakan.

```
$ pakai setup
```

#### 4.1.1 Yang Dilakukan oleh `pakai setup`

1. **Deteksi penyedia:** memeriksa jalur berikut:
  - Claude: `~/.claude/stats-cache.json`
  - OpenCode Go: `~/.local/share/opencode/opencode-stable.db` atau `opencode.db`
  - OpenAI Codex: `~/.codex/auth.json`
2. **Memulai daemon** jika belum berjalan.
3. **Membuat unit systemd** di `~/.config/systemd/user/pakai.service`.
4. **Menampilkan cuplikan konfigurasi** untuk tmux dan Waybar.

### 4.2 Memulai Daemon

Daemon adalah proses latar belakang yang mengambil dan menyimpan data penggunaan. Tanpa daemon, perintah-perintah PakAI tidak akan menampilkan data.

#### 4.2.1 Memulai daemon secara manual

```
$ pakai daemon start
Daemon started (port 7731)
```

```
$ pakai setup

x Not found: claude (/home/dhs/.claude/stats-cache.json)
✓ Detected: opencode at /home/dhs/.local/share/opencode/opencode-stable.db
✓ Detected: openai at /home/dhs/.codex/auth.json

Daemon: already running
Systemd unit: written to /home/dhs/.config/systemd/user/pakai.service
Enable with: systemctl --user enable --now pakai

tmux – add to your ~/.tmux.conf:
set -g status-right "#(/home/dhs/.asdf/installs/golang/1.26.3/bin/pakai tmux)"
set -g status-interval 30

waybar – add to your config.jsonc:
"custom/pakai": {
  "exec": "pakai waybar",
  "return-type": "json",
  "interval": 30
}

--- Live Preview ---
tmux:      8% | ? | $0 | $2.5
waybar: {"text": "\u003cspan foreground='#a6e3a1'\u003e 8%\u003c/span\u003e
| \u003cspan foreground='#fab387'\u003e ?\u003c/span\u003e | \u003cspan
foreground='#a6e3a1'\u003e $0\u003c/span\u003e | \u003cspan
foreground='#a6e3a1'\u003e $2.5\u003c/span\u003e","tooltip":" claude\n
5h ████████ 8% resets in 4h 24m\n w
██████ 8% resets in 21h 54m\nopenai: Get
\"https://chatgpt.com/backend-api/wham/usage\": tls: failed to verify
certificate: x509: certificate signed by unknown authority\n
opencode/openai: $0.00 this month\n opencode/opencode-go: $2.51 this
month","class":"error","percentage":8}
```

Gambar 4.1: Output pakai setup: deteksi penyedia dan konfigurasi otomatis

## 4.2.2 Memeriksa status daemon

```
$ pakai daemon status
```

```
$ pakai daemon status

Daemon: running (PID 13936, port 7731, uptime 52s)
```

Gambar 4.2: Output pakai daemon status

## 4.2.3 Menghentikan daemon

```
$ pakai daemon stop
```

## 4.3 Konfigurasi Autostart dengan Systemd

Agar daemon berjalan otomatis saat login, aktifkan unit systemd yang dibuat oleh pakai setup:

```
$ systemctl --user enable --now pakai
$ systemctl --user status pakai
```

Isi unit systemd yang dibuat secara otomatis:

```
[Unit]
Description=PakAI usage tracker daemon
After=default.target

[Service]
ExecStart=/home/user/.local/bin/pakai daemon start --
    foreground
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

### Tips

Gunakan `journalctl -user -u pakai -f` untuk melihat log daemon secara langsung (*real-time*).

## 4.4 Autentikasi Penyedia

### 4.4.1 Claude

Claude menggunakan OAuth yang dikelola oleh Claude Code CLI. Jalankan:

```
$ claude login
```

Kredensial disimpan di `~/.claude/.credentials.json`.

### 4.4.2 OpenAI Codex

```
$ codex login
```

Kredensial disimpan di `~/.codex/auth.json`.

### 4.4.3 OpenCode Go

OpenCode Go tidak memerlukan login tambahan. PakAI membaca langsung dari database SQLite yang dibuat oleh OpenCode. Pastikan OpenCode sudah pernah digunakan minimal satu kali sehingga database terbentuk di:

```
~/.local/share/opencode/opencode-stable.db
```





# Bab 5

## Penggunaan Dasar

### 5.1 Mengapa Memantau Penggunaan AI?

Berlangganan lebih dari satu layanan AI berarti mengelola beberapa jendela waktu kuota yang berbeda: Claude membatasi pesan per 5 jam dan mingguan, OpenAI Codex membatasi per 5 jam dan mingguan, sementara OpenCode mencatat biaya token secara langsung. Tanpa pemantauan, ada tiga masalah yang sering dialami:

- **Kuota habis di tengah pekerjaan.** Claude atau Codex berhenti merespons tanpa peringatan sebelumnya, memutus alur kerja saat sedang fokus.
- **Biaya tak terduga.** Token yang digunakan melalui OpenCode langsung ditagih; tanpa visibilitas, tagihan bulanan bisa mengejutkan.
- **Konteks berpindah-pindah.** Memeriksa kuota berarti membuka dasbor web masing-masing penyedia, memecah konsentrasi dari terminal.

PakAI menyelesaikan ketiga masalah ini dengan satu perintah di terminal. Status semua penyedia tersedia setiap saat — di status bar tmux, di panel Waybar, maupun di dashboard TUI layar penuh — tanpa harus meninggalkan lingkungan kerja.

#### Tips

PakAI cocok untuk siapa saja yang menggunakan dua atau lebih layanan AI berbayar dan ingin tahu kapan harus beralih penyedia atau mengistirahatkan sesi agar kuota tidak terbuang.

### 5.2 Memeriksa Status Penggunaan

Perintah utama untuk melihat ringkasan penggunaan semua penyedia:

```
$ pakai status
```

Output menampilkan:

- Nama penyedia dan label yang dikonfigurasi.
- Persentase atau biaya penggunaan per jendela waktu (5j, mingguan, bulanan).

```
$ pakai status

openai          0.00          (no limit set)
claude          5h: 7% (resets in 4h 25m) | w: 8% (resets in 21h 55m)
opencode/openai $0.00        (no limit set)
opencode/opencode-go $2.51    (no limit set)

4 providers · refreshed 19s ago
```

Gambar 5.1: Output `pakai status`: ringkasan semua penyedia aktif

- Waktu reset atau sisa waktu jendela.
- Jumlah penyedia aktif dan waktu pembaruan terakhir.

### 5.2.1 Output JSON

Tambahkan flag `-json` untuk mendapatkan output dalam format JSON mentah:

```
$ pakai status --json
```

## 5.3 Dashboard TUI Interaktif

Dashboard TUI menampilkan tampilan penuh layar dengan pembaruan langsung:

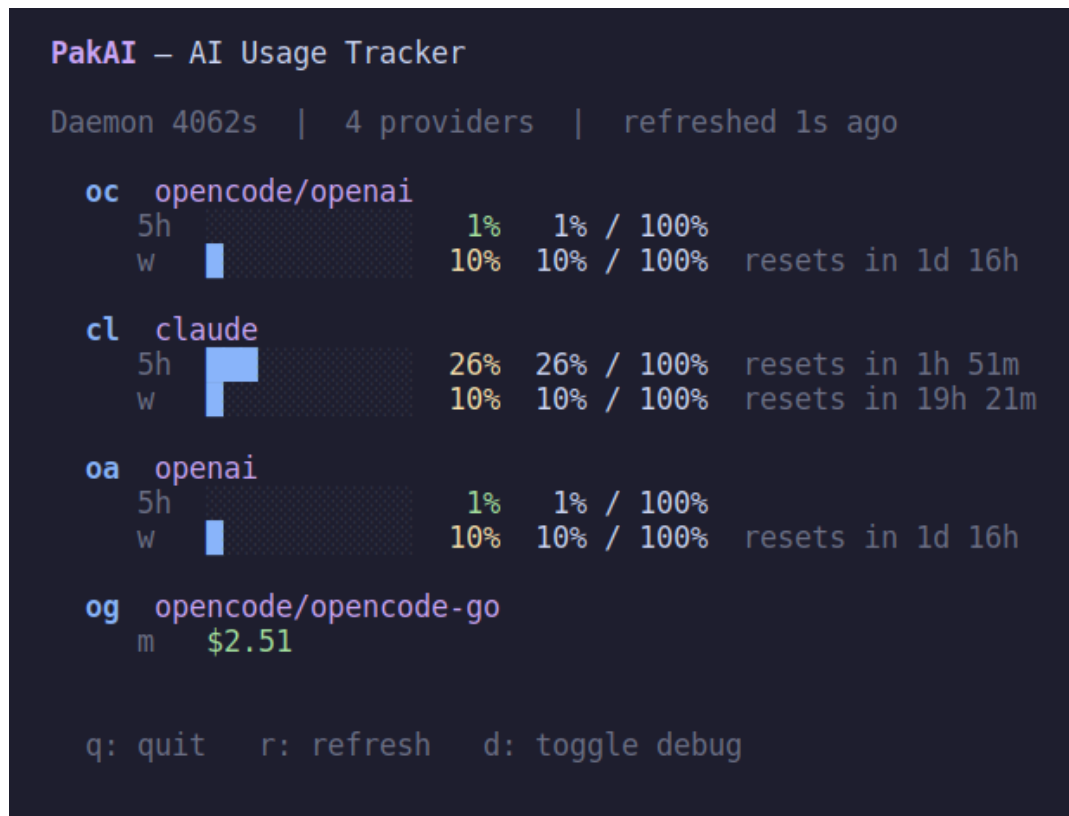
```
$ pakai dashboard
```

Dashboard diperbarui secara otomatis setiap kali daemon menerima data baru dari penyedia. Tampilan dibagi per penyedia, masing-masing menampilkan baris untuk setiap jendela waktu yang aktif.

### 5.3.1 Membaca tampilan dashboard

Setiap baris di dalam dashboard mengikuti format berikut:

| Kolom         | Keterangan                                                            |
|---------------|-----------------------------------------------------------------------|
| Ikon / kode   | Ikon Nerd Font penyedia; fallback dua huruf bila font tidak tersedia. |
| Nama penyedia | Label penyedia, termasuk sub-penyedia OpenCode.                       |
| Jendela waktu | 5h (5 jam), w (mingguan), m (bulanan).                                |
| Progress bar  | Bar 12 segmen menunjukkan proporsi penggunaan saat ini.               |
| Persentase    | Angka persentase atau biaya dalam dolar (\$).                         |
| Waktu reset   | Sisa waktu sebelum kuota jendela direset.                             |



Gambar 5.2: Dashboard TUI PakAI: empat penyedia aktif dengan progress bar dan sisa waktu reset per jendela waktu

### 5.3.2 Kode warna

PakAI menggunakan tiga warna untuk persentase penggunaan:

- **Hijau** — penggunaan di bawah ambang peringatan (aman).
- **Kuning** — mendekati batas; pertimbangkan untuk beralih penyedia atau mengurangi frekuensi permintaan.
- **Merah** — mendekati atau melebihi batas; kuota hampir habis.

Ambang kuning dan merah dapat dikonfigurasi melalui `pakai config set` (lihat Bab 7).

### 5.3.3 Kapan dashboard berguna

Dashboard layar penuh paling berguna dalam skenario berikut:

- **Sebelum sesi panjang:** periksa penyedia mana yang masih punya kuota cukup sebelum memulai pekerjaan berat.
- **Saat jendela waktu hampir reset:** pantau kolom *resets in* untuk memilih waktu terbaik memulai ulang sesi.
- **Membandingkan biaya:** lihat akumulasi biaya dolar OpenCode berdampingan dengan kuota pesan Claude dan Codex.

- **Debugging penyedia:** tekan `d` untuk melihat JSON mentah yang dikirim setiap penyedia, berguna saat ada anomali data.

**Tombol keyboard:**

| Tombol | Fungsi                                                  |
|--------|---------------------------------------------------------|
| q      | Keluar dari dashboard                                   |
| r      | Muat ulang data secara manual                           |
| d      | Aktifkan/nonaktifkan mode debug (tampilkan JSON mentah) |

#### Informasi

Jika daemon belum berjalan, jalankan `pakai daemon start` terlebih dahulu sebelum membuka dashboard.

## 5.4 Output untuk Tmux

Perintah `pakai tmux` menghasilkan string ringkas untuk status bar tmux:

```
$ pakai tmux
7% | 8% | $0.00
```

Output menggunakan ikon Nerd Font untuk setiap penyedia. Pastikan terminal menggunakan font yang mendukung Nerd Font.

## 5.5 Output untuk Waybar

Perintah `pakai waybar` menghasilkan JSON untuk modul kustom Waybar:

```
$ pakai waybar
```

```
{
  "text": "<span foreground='#a6e3a1'>\udb81\udea9 8%</span> | <span
foreground='#fab387'>\udb86\udc86 ?</span> | <span
foreground='#a6e3a1'>\udb86\udc86 $0</span> | <span
foreground='#a6e3a1'>\udb81\ude26 $2.5</span>",
  "tooltip": "\udb81\udea9 claude\n 5h
\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591 8% resets in 4h
24m\n w \ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591\ud2591 8%
resets in 21h 54m\nopenai: Get \"https://chatgpt.com/backend-api/wham/usage\":
tls: failed to verify certificate: x509: certificate signed by unknown
authority\n\udb86\udc86 opencode/openai: $0.00 this month\n\udb81\ude26
opencode/opencode-go: $2.51 this month",
  "class": "error",
  "percentage": 8
}
```

Gambar 5.3: Output JSON dari `pakai waybar`

Format output JSON:

```
{
  "text": "<teks tampilan>",
  "tooltip": "<detail per penyedia>",
  "class": "ok|warning|critical|error",
  "percentage": 0
}
```

## 5.6 Men-debug Penyedia

Untuk memeriksa detail data satu penyedia tertentu:

```
$ pakai provider debug claude
$ pakai provider debug opencode
$ pakai provider debug openai
```

```
$ pakai provider debug claude

Provider: claude
Scanning: /home/dhs/.claude/stats-cache.json
  x Not found: /home/dhs/.claude/stats-cache.json
Status: ok
Used: 0.00
  - 5h: 8%
  - weekly: 8%
```

Gambar 5.4: Output pakai provider debug claude

## 5.7 Completion Shell

Aktifkan pelengkapan otomatis perintah (tab completion) untuk shell Anda:

```
# Bash
$ pakai completion bash >> ~/.bashrc

# Zsh
$ pakai completion zsh >> ~/.zshrc

# Fish
$ pakai completion fish > ~/.config/fish/completions/pakai.fish
```



# Bab 6

## Integrasi

### 6.1 Integrasi Tmux

#### 6.1.1 Konfigurasi dasar

Tambahkan baris berikut ke berkas `~/.tmux.conf`:

```
# Tampilkan status PakAI di status bar kanan
set -g status-right "#(pakai tmux)"
set -g status-interval 30

# Opsional: perpanjang lebar status bar
set -g status-right-length 100
```

Muat ulang konfigurasi tmux tanpa harus me-restart:

```
$ tmux source-file ~/.tmux.conf
```

#### 6.1.2 Konfigurasi dengan tmux-powerline

Jika menggunakan *tmux-powerline* atau tema kustom, sesuaikan `status-right` dengan format yang digunakan tema Anda.

##### Tips

Jalankan `pakai setup tmux` untuk mendapatkan cuplikan konfigurasi tmux yang siap tempel (*ready-to-paste*).

#### 6.1.3 Persyaratan font

Ikon penyedia menggunakan karakter **Nerd Font**. Pastikan terminal dan tmux menggunakan font Nerd Font seperti:

- JetBrains Mono Nerd Font
- Fira Code Nerd Font
- DejaVu Sans Mono Nerd Font

Jika ikon tidak muncul, ikon akan digantikan dengan dua huruf pertama nama penyedia (misalnya `cl` untuk Claude).

## 6.2 Integrasi Waybar

### 6.2.1 Konfigurasi modul

Tambahkan modul kustom ke berkas konfigurasi Waybar (biasanya `~/.config/waybar/config.jsonc`):

```
"custom/pakai": {
  "exec": "pakai waybar",
  "return-type": "json",
  "interval": 30,
  "tooltip": true,
  "format": "{}",
  "on-click": "pakai dashboard"
}
```

Tambahkan `"custom/pakai"` ke array `modules-right` (atau `modules-left/modules-center` sesuai preferensi):

```
"modules-right": [
  "custom/pakai",
  "clock",
  "battery"
]
```

### 6.2.2 Konfigurasi CSS

Tambahkan style ke berkas CSS Waybar (`style.css`) untuk pewarnaan berdasarkan status:

```
#custom-pakai {
  padding: 0 8px;
  color: #cdd6f4;
}

#custom-pakai.ok {
  color: #a6e3a1;
}

#custom-pakai.warning {
  color: #f9e2af;
}

#custom-pakai.critical {
  color: #f38ba8;
}

#custom-pakai.error {
```



```
color: #fab387;  
}
```

### 6.2.3 Tooltip

Waybar menampilkan tooltip berisi rincian penggunaan per jendela waktu ketika kursor diarahkan ke modul PakAI. Pastikan "tooltip": true sudah dikonfigurasi di modul.

#### Tips

Jalankan `pakai setup waybar` untuk mendapatkan cuplikan konfigurasi Waybar yang lengkap, termasuk JSON modul dan CSS.

## 6.3 Integrasi Shell Prompt

PakAI juga dapat ditampilkan di prompt shell. Contoh untuk Zsh dengan **Starship**:

```
# ~/.config/starship.toml  
[custom.pakai]  
command = "pakai tmux"  
when = "pakai daemon status"  
format = "[$output]($style) "  
style = "green"
```



## Bab 7

# Konfigurasi Lanjutan

### 7.1 Berkas Konfigurasi

PakAI menyimpan konfigurasi di:

- `$XDG_CONFIG_HOME/pakai/config.toml`: jika variabel tersebut diset.
- `~/.config/pakai/config.toml`: lokasi default.

Berkas dibuat secara otomatis saat pertama kali menyimpan konfigurasi.

### 7.2 Melihat Semua Konfigurasi

```
$ pakai config list
```

```
$ pakai config list

daemon.port = 7731
daemon.poll_interval = 30
display.separator = " | "
display.order = []
thresholds.warning = 50
thresholds.critical = 80
```

Gambar 7.1: Output `pakai config list`: semua konfigurasi aktif

## 7.3 Mengubah Konfigurasi

Gunakan `pakai config set kunci nilai` untuk mengubah nilai:

```
$ pakai config set daemon.port 7732
$ pakai config get daemon.port
```

## 7.4 Referensi Kunci Konfigurasi

### 7.4.1 Konfigurasi Daemon

| Kunci                | Tipe    | Default | Keterangan                                |
|----------------------|---------|---------|-------------------------------------------|
| daemon.port          | integer | 7731    | Port HTTP daemon (1–65535)                |
| daemon.poll_interval | integer | 30      | Interval polling dalam detik ( $\geq 1$ ) |

```
$ pakai config set daemon.poll_interval 15
$ pakai config set daemon.port 7732
```

### 7.4.2 Konfigurasi Tampilan

| Kunci             | Tipe   | Default | Keterangan                    |
|-------------------|--------|---------|-------------------------------|
| display.separator | string |         | Pemisah antar penyedia        |
| display.order     | string | (auto)  | Urutan penyedia, dipisah koma |

```
$ pakai config set display.separator " - "
$ pakai config set display.order "claude,openai"
```

### 7.4.3 Konfigurasi Ambang Batas

| Kunci               | Tipe    | Default | Keterangan                 |
|---------------------|---------|---------|----------------------------|
| thresholds.warning  | integer | 50      | Ambang peringatan (0–100%) |
| thresholds.critical | integer | 80      | Ambang kritis (0–100%)     |

```
$ pakai config set thresholds.warning 60
$ pakai config set thresholds.critical 85
```

### 7.4.4 Konfigurasi Per Penyedia

Setiap penyedia dapat dikonfigurasi dengan format `provider.id.kunci`:

| Kunci                            | Tipe   | Default | Keterangan                         |
|----------------------------------|--------|---------|------------------------------------|
| <code>provider.id.enabled</code> | bool   | true    | Aktifkan/nonaktifkan penyedia      |
| <code>provider.id.label</code>   | string | (id)    | Label tampilan kustom              |
| <code>provider.id.limit</code>   | float  | 0       | Batas biaya dalam USD ( $\geq 0$ ) |

ID penyedia yang tersedia: `claude`, `opencode`, `openai`, `opencode/openai`, `opencode/opencode-go`.

```
# Atur label kustom untuk Claude
$ pakai config set provider.claude.label "Klaude"

# Atur batas bulanan untuk OpenCode Go
$ pakai config set provider.opencode-go.limit 60

# Nonaktifkan penyedia OpenAI sementara
$ pakai config set provider.openai.enabled false
```

## 7.5 Contoh Berkas Konfigurasi Lengkap

```
[daemon]
port = 7731
poll_interval = 30

[display]
separator = " | "
order = "claude,opencode/opencode-go,openai"

[thresholds]
warning = 50
critical = 80

[provider.claude]
enabled = true
label = "Claude"

[provider.openai]
enabled = true
label = "Codex"

[provider.opencode]
enabled = true
label = "OpenCode"
```

```
[provider."opencode/opencode-go"]
enabled = true
limit = 60.0
```

## 7.6 Mock Provider (untuk Pengujian)

PakAI mendukung penyedia palsu (*mock*) untuk keperluan pengujian tampilan:

```
# Buat mock dengan 75% penggunaan
$ pakai provider mock test-ai --used 75 --limit 100 --unit messages

# Hapus mock
$ pakai provider unmock test-ai
```

# Bab 8

## Referensi Perintah

Berikut adalah daftar lengkap semua perintah yang tersedia di PakAI.

### 8.1 Perintah Utama

| Perintah                            | Keterangan                                                  |
|-------------------------------------|-------------------------------------------------------------|
| <code>pakai version</code>          | Menampilkan versi PakAI yang terinstal.                     |
| <code>pakai status</code>           | Menampilkan ringkasan penggunaan semua penyedia.            |
| <code>pakai status -json</code>     | Menampilkan data penggunaan dalam format JSON mentah.       |
| <code>pakai tmux</code>             | Menghasilkan string ringkas untuk status bar tmux.          |
| <code>pakai waybar</code>           | Menghasilkan JSON untuk modul kustom Waybar.                |
| <code>pakai dashboard</code>        | Membuka antarmuka TUI interaktif dengan pembaruan langsung. |
| <code>pakai completion shell</code> | Menghasilkan skrip <i>tab completion</i> (bash/zsh/fish).   |

### 8.2 Perintah Daemon

| Perintah                                    | Keterangan                                         |
|---------------------------------------------|----------------------------------------------------|
| <code>pakai daemon start</code>             | Menjalankan daemon di latar belakang.              |
| <code>pakai daemon start -foreground</code> | Menjalankan daemon di latar depan (untuk systemd). |
| <code>pakai daemon stop</code>              | Menghentikan daemon yang berjalan.                 |
| <code>pakai daemon status</code>            | Menampilkan status daemon (PID, port, up-time).    |

## 8.3 Perintah Setup

| Perintah                        | Keterangan                                                  |
|---------------------------------|-------------------------------------------------------------|
| <code>pakai setup</code>        | Deteksi penyedia, buat unit systemd, tampilkan konfigurasi. |
| <code>pakai setup tmux</code>   | Tampilkan cuplikan konfigurasi tmux.                        |
| <code>pakai setup waybar</code> | Tampilkan cuplikan konfigurasi Waybar dan CSS.              |

## 8.4 Perintah Konfigurasi

| Perintah                                         | Keterangan                                |
|--------------------------------------------------|-------------------------------------------|
| <code>pakai config list</code>                   | Menampilkan semua konfigurasi aktif.      |
| <code>pakai config get <i>kunci</i></code>       | Menampilkan nilai satu kunci konfigurasi. |
| <code>pakai config set <i>kunci nilai</i></code> | Mengatur nilai kunci konfigurasi.         |

## 8.5 Perintah Penyedia

| Perintah                                     | Keterangan                                                                    |
|----------------------------------------------|-------------------------------------------------------------------------------|
| <code>pakai provider debug <i>id</i></code>  | Menampilkan data debug untuk penyedia tertentu.                               |
| <code>pakai provider mock <i>id</i></code>   | Membuat penyedia palsu untuk pengujian.                                       |
| <code>-used <i>float</i></code>              | Nilai penggunaan (default: 0).                                                |
| <code>-limit <i>float</i></code>             | Nilai batas, 0 = tanpa batas (default: 0).                                    |
| <code>-unit <i>string</i></code>             | Satuan: <code>messages</code> , <code>tokens</code> , atau <code>usd</code> . |
| <code>pakai provider unmock <i>id</i></code> | Menghapus penyedia palsu.                                                     |

## 8.6 HTTP API Daemon

Daemon mengekspos API HTTP lokal di `http://127.0.0.1:7731`.

| Method | Endpoint                       | Keterangan                                          |
|--------|--------------------------------|-----------------------------------------------------|
| GET    | <code>/health</code>           | Status daemon, uptime, jumlah koneksi.              |
| GET    | <code>/status</code>           | Data penggunaan semua penyedia (JSON).              |
| GET    | <code>/events</code>           | <i>Server-Sent Events</i> untuk pembaruan langsung. |
| PUT    | <code>/mock/<i>name</i></code> | Memasukkan data mock penyedia.                      |



| Method | Endpoint            | Keterangan                    |
|--------|---------------------|-------------------------------|
| DELETE | /mock/ <i>name</i>  | Menghapus data mock penyedia. |
| GET    | /debug/ <i>name</i> | Debug data satu penyedia.     |

```
# Contoh mengakses API langsung
$ curl -s http://127.0.0.1:7731/health | python3 -m json.tool
$ curl -s http://127.0.0.1:7731/status | python3 -m json.tool
```



# Bab 9

## Pemecahan Masalah

### 9.1 Masalah Instalasi

#### 9.1.1 Perintah pakai tidak ditemukan

Gejala: bash: pakai: command not found

Penyebab: Direktori instalasi tidak ada di \$PATH.

Solusi:

```
# Cari lokasi binary
$ which pakai || find ~/go ~/go/bin ~/.local/bin -name pakai 2>/dev/null

# Tambahkan ke PATH (sesuaikan path yang ditemukan)
$ echo 'export PATH="$PATH:$HOME/go/bin"' >> ~/.bashrc
$ source ~/.bashrc
```

Jika menggunakan asdf:

```
$ asdf reshim go lang
```

#### 9.1.2 Build gagal: versi Go terlalu lama

Gejala: requires go >= 1.21

Solusi: Perbarui Go ke versi 1.21 atau lebih baru.

```
$ go version          # cek versi saat ini
$ asdf install go lang latest # update via asdf
```

### 9.2 Masalah Daemon

#### 9.2.1 Daemon gagal dimulai: port sudah digunakan

Gejala: port 7731 is already in use by another process

Solusi:

```
# Cek proses yang menggunakan port
$ lsof -i :7731 || ss -tlnp | grep 7731
```

```
# Hentikan daemon lama
$ pakai daemon stop

# Atau gunakan port berbeda
$ pakai config set daemon.port 7732
$ pakai daemon start
```

### 9.2.2 Daemon berhenti tiba-tiba

**Solusi:** Periksa log systemd:

```
$ journalctl --user -u pakai -n 50
```

### 9.2.3 Status daemon menunjukkan data lama (*stale*)

**Penyebab:** Interval polling terlalu jarang atau koneksi internet bermasalah.

**Solusi:**

```
# Perkecil interval polling
$ pakai config set daemon.poll_interval 15
$ pakai daemon stop
$ pakai daemon start
```

## 9.3 Masalah Penyedia Claude

### 9.3.1 Claude stats cache tidak ditemukan

**Gejala:** Claude stats cache not found: ensure Claude Code is running

**Penyebab:** Berkas ~/.claude/stats-cache.json tidak ada.

**Solusi:** Pastikan Claude Code CLI telah diinstal dan pernah dijalankan:

```
$ ls ~/.claude/stats-cache.json
$ claude --version # verifikasi instalasi
```

### 9.3.2 Kredensial Claude tidak ditemukan

**Gejala:** Claude credentials not found: ensure Claude Code is installed

**Solusi:**

```
$ claude login
```

### 9.3.3 Persentase Claude tidak tersedia

**Gejala:** live percentage unavailable

**Penyebab:** API OAuth gagal, tetapi data lokal tersedia. PakAI tetap menampilkan data cache.

**Solusi:** Periksa koneksi internet. Jika masalah berlanjut:

```
$ claude logout && claude login
```

## 9.4 Masalah Penyedia OpenAI Codex

### 9.4.1 Kredensial Codex tidak ditemukan

**Gejala:** Codex credentials not found: ensure Codex is installed

**Solusi:**

```
$ codex login
$ ls ~/.codex/auth.json # verifikasi
```

### 9.4.2 Permintaan penggunaan Codex gagal

**Gejala:** codex usage request failed: status 401

**Penyebab:** Token akses kedaluwarsa.

**Solusi:**

```
$ codex logout && codex login
```

### 9.4.3 Kesalahan sertifikat TLS

**Gejala:** tls: failed to verify certificate: x509: certificate signed by unknown authority

**Penyebab:** Proxy perusahaan atau VPN yang melakukan *TLS inspection*.

**Solusi:** Tambahkan sertifikat CA ke kepercayaan sistem, atau nonaktifkan sementara penyedia yang bermasalah:

```
$ pakai config set provider.openai.enabled false
```

## 9.5 Masalah Penyedia OpenCode

### 9.5.1 Database OpenCode tidak ditemukan

**Gejala:** opencode database not found: ensure opencode is installed

**Penyebab:** OpenCode belum pernah digunakan atau database belum terbentuk.

**Solusi:** Gunakan OpenCode minimal satu kali sehingga database terbentuk:

```
$ ls ~/.local/share/opencode/
# Harus ada: opencode-stable.db atau opencode.db
```

### 9.5.2 Database OpenCode sibuk (*locked*)

**Gejala:** database is busy: another process may be writing

**Penyebab:** OpenCode sedang aktif menulis ke database.

**Solusi:** Tunggu beberapa detik, lalu coba lagi. PakAI akan berhasil pada polling berikutnya secara otomatis.

## 9.6 Masalah Tampilan

### 9.6.1 Ikon tidak muncul di tmux

**Penyebab:** Font terminal tidak mendukung Nerd Font.

**Solusi:** Instal dan atur font Nerd Font di terminal dan tmux. Contoh untuk terminal berbasis X11:

```
# Pasang font (Ubuntu/Debian)
$ sudo apt install fonts-jetbrains-mono

# Atau unduh dari nerdfonts.com
```

### 9.6.2 Waybar tidak memperbarui data

**Penyebab:** Interval Waybar atau daemon terlalu panjang.

**Solusi:** Sesuaikan interval di konfigurasi Waybar:

```
"custom/pakai": {
  "interval": 15
}
```

## 9.7 Diagnosis Umum

Jika masalah tidak tercantum di atas, gunakan langkah diagnosis berikut:

```
# 1. Cek status daemon
$ pakai daemon status

# 2. Debug penyedia yang bermasalah
$ pakai provider debug claude
$ pakai provider debug opencode
$ pakai provider debug openai

# 3. Cek log daemon
$ journalctl --user -u pakai -n 100

# 4. Cek koneksi ke daemon
$ curl -s http://127.0.0.1:7731/health

# 5. Reset daemon
$ pakai daemon stop && pakai daemon start
```